Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

WAVELET-FEATURE BASED BEARING FAULT DIAGNOSIS FOR PREDICTIVE MAINTENANCE USING RANDOM FOREST AND XGBOOST

Submitted in partial fulfilment of the requirements in the subject of
Advanced Digital Signal Processing (ADSP) Mini Project
(DJS23ECPE723 / DJS23ELPE723)

by

SHRUTIK ACHARYA (E149)
SIDDHAANT PANDIT (E151)
SOHAM BHOSALE (E155)

Under the guidance of **Prof. Ranjushree Pal**

Electronics & Telecommunication Engineering
Dwarkadas J. Sanghvi College of Engineering
2026 – 2027

Table of Contents

CHAPTER NO.	SECTION NO.	TOPIC	PAGE NO.
		Certificate	i
		Abstract	1
1.		**Introduction**	1
	1.1	Problem Statement	1
	1.2	Overview	1
	1.3	Objectives	1
	1.4	Literature Review	2
2.		**Methodology**	3
	2.1	System Overview	3
	2.2	Overall Workflow	3
	2.3	System Architecture Diagram	4
	2.4	System Components	4
	2.5	Data Pre-Processing	5
	2.6	Model Implementation	5
3.		**Results and Discussion**	6
	3.1	Classification Accuracy on the Test Set	6
	3.2	Per-Class Performance	6
	3.3	Feature-Set and Wavelet Ablation	7
	3.4	Discussion	7
	3.5	Conclusion	8
	3.6	Future Scope	8
		References	8
A		**Appendix A: Python Code**	10

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Electronics & Telecommunication Engineering

Certificate

This is to certify that the project entitled **"WAVELET-FEATURE BASED BEARING FAULT DIAGNOSIS FOR PREDICTIVE MAINTENANCE USING RANDOM FOREST AND**

1. **SHRUTIK ACHARYA (Roll No.** E149**)**
2. **SIDDHAANT PANDIT (Roll No.** E151**)**
3. **SOHAM BHOSALE (Roll No.** E155**)**

submitted in partial fulfilment of the requirements in the subject of **AdvancedDigital SignalProcessing(ADSP)MiniProject(DJS23ECPE723/DJS23ELPE723)**in Electronics & Telecommunication Engineering.

Teacher-in-charge
Prof. Ranjushree Pal

Head of Department
Dr. Amit Deshmukh

Abstract

Unplanned failure of rotating machinery is costly, and rolling-element bearings areamongitsmostcommoncauses.Afaultybearingproducesshort,repetitiveimpulseswhose energyliesindifferentfrequencybands,andaplainFouriertransformcannotlocalise theseintime.Thisprojectbuildsalightweightpredictive-maintenancepipelineonthepublic Case Western Reserve University (CWRU) bearing dataset: vibration records are cut into 2048-sample windows, decomposed by a 5-level Discrete Wavelet Transform (db4), describedby37sub-bandenergy,entropyandstatisticalfeatures,andclassifiedbya RandomForestandXGBoostinto10healthstates(normal,plusinner-race,balland outer-race faults at three sizes). To avoid optimistic results, models are trained on three motor loads and tested on an unseen fourth load. On this protocol the best model reaches 99.1% accuracy (macro F1 0.991) against 96.7% for time-domain features alone. The implementation uses Python, PyWavelets and scikit-learn / XGBoost in Google Colab.

1 Introduction

1.1 Problem Statement

A bearing fault should be detected and identified, by location and severity, before itcauses machine breakdown. The vibration signal is non-stationary and noisy, so a method is needed that localises fault-related transients in both time and frequency and then classifies the machine state automatically, without a large GPU-trained model.

1.2 Overview

Predictive maintenance replaces fixed-interval servicing with condition-baseddecisions. Classical diagnosis uses hand-crafted time, frequency and time-frequency features with a machine-learning classifier, whereas deep networks learn features from raw signals but need more data and compute and are harder to explain. The Fourier transform loses time information and the Short-Time Fourier Transform has a fixed resolution. The wavelet transform gives multi-resolution analysis, with short windows at high frequencies to catch impulses and long windows at low frequencies. This project extracts DWT filter-bank features from CWRU vibration data and compares Random Forest and XGBoost classifiers.

1.3 Objectives

The main objective is to build a fast and explainable bearing fault classifier basedon wavelet features. Specific objectives include:

- To segment CWRU drive-end vibration records into windows labelled with 10 health states
- To apply multi-resolution DWT analysis (Haar and Daubechies wavelets) and extract a 37-value sub-band energy, entropy and statistics vector
- To compare Random Forest, XGBoost and an SVM baseline using a cross-load protocol
- To quantify the benefit of wavelet over time-domain features and comparemother wavelets
- To evaluate with accuracy, macro F1, a per-class report, a confusion matrix and feature importance

1.4 Literature Review

Table 1 summarises the dataset paper [1], benchmarking practice [2], wavelet theory [3], [4], a fault-diagnosis survey [5] and the classifiers used [6].

Ref.	Title and authors	Novel contribution	Year and venue
[1]	**CWRU Bearing Data Center** *Case Western Reserve University*	Public benchmark of vibration data from healthy and seeded-fault bearings (inner race, ball, outer race) at several loads and faultsizes.	Online dataset, Case Western Reserve University
[2]	**Rolling element bearing diagnostics using the CWRU data: A benchmark study** *W. A. Smith, R. B. Randall*	Examines CWRU records in detail and shows some fault classes are much harder than often reported, motivating careful protocols.	2015, Mech. Systems and Signal Processing, Vol. 64-65
[3]	**A theory for multiresolution signal decomposition: the wavelet representation** *S. G. Mallat*	Introduces multi-resolution analysis and the filter-bank algorithm behind the DWT.	1989, IEEE Trans. PAMI, Vol. 11, No. 7
[4]	**Ten Lectures on Wavelets** *I. Daubechies*	Constructs compactly supported orthogonal wavelets (the dbfamily).	1992, SIAM
[5]	**Applications of machine learning to machine fault diagnosis: A review and roadmap** *Y. Lei et al.*	Surveys feature-based machine learning and deep learning for fault diagnosis.	2020, Mech. Systems and Signal Processing, Vol. 138

Ref.	Title and authors	Novel contribution	Year and venue
[6]	**Random forests** (L. Breiman); **XGBoost: A scalable tree boosting system** (T. Chen, C. Guestrin)	Bagged trees with feature importance; regularised gradient-boosted trees.	2001, Machine Learning; 2016, ACM KDD

Table 1: Summary of the literature reviewed

Deep networks can reach very high accuracy on CWRU but need more data and compute and are harder to explain. A wavelet-feature plus tree-ensemble pipeline, as built here, trainsinsecondsonaCPU,showswhichsub-bandsmatterandgivesaclearbaseline against which deep models can be benchmarked.

2 Methodology

2.1 System Overview

The system is a modular supervised pipeline that turns a raw vibration record intoa machine-health label. Each stage (windowing, denoising, wavelet decomposition, feature extraction and classification) can be inspected or replaced, for example by a wavelet-packet front end or a 1-D CNN.

2.2 Overall Workflow

CWRU 12 kHz drive-end records are loaded from .mat files and cut into windows of2048 samples with 50% overlap. Each window is optionally denoised by soft wavelet thresholding and decomposed by a 5-level DWT. For every sub-band the mean absolute value, standard deviation, kurtosis, skewness and peak are computed, together with the relative sub-band energies and the wavelet entropy, giving a 37-value vector. Random Forest, XGBoost and an RBF-SVM are trained on motor loads 0, 1 and 2 hp and tested on 3 hp, and each load is also held out in turn. Results are reported as accuracy, macro F1, a per-class report and a confusion matrix.

2.3 System Architecture Diagram

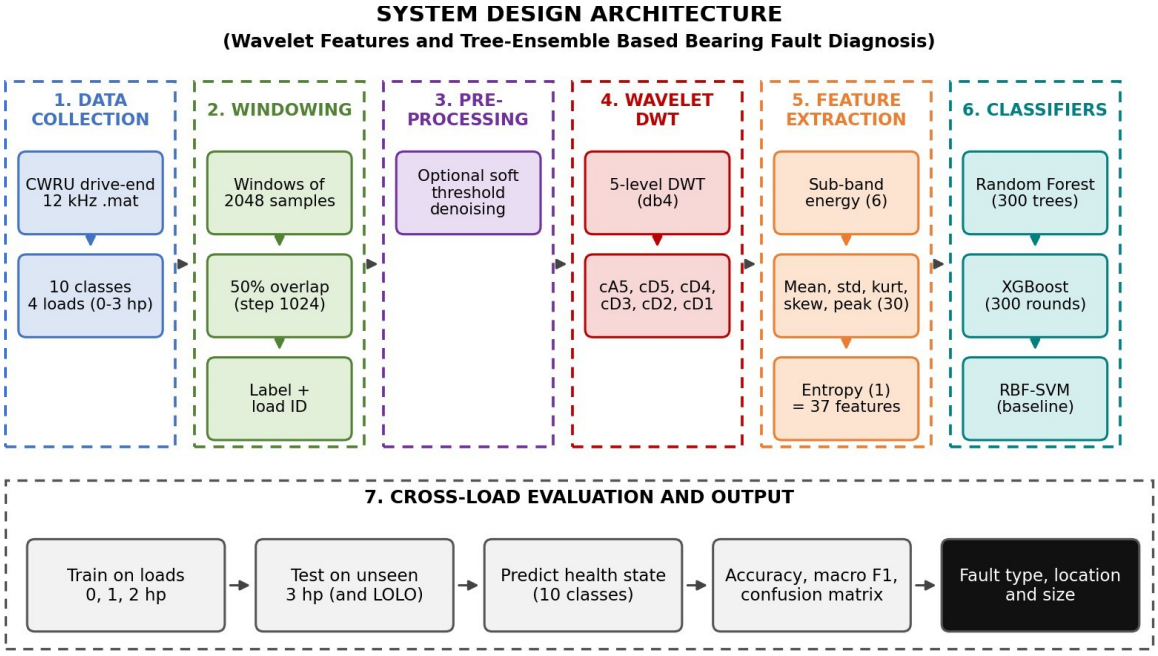


Figure 1: System architecture of the wavelet-feature bearing fault diagnosis pipeline

2.4 System Components

- **Data Input:** CWRU drive-end accelerometer data at 12 kHz, motor loads 0–3 hp (about 1797–1730 rpm).
- **Classes:** normal, plus inner-race, ball and outer-race (6 o'clock) faults at 0.007, 0.014 and 0.021 inch fault diameter (10 classes).
- **Segmentation:** 2048-sample windows (about 0.17 s) with step 1024.
- **Signal Processing:** optional soft-threshold denoising and a 5-level DWT withHaar,db2, db4 or db8.
- **Feature Extraction:** 37 wavelet features per window; 8 time-domain features as a baseline.
- **Classification:** Random Forest, XGBoost and an RBF-SVM baseline.
- **Evaluation:** cross-load test, accuracy, macro F1, confusion matrix and feature importance.

2.5 Data Pre-Processing

The CWRU Bearing Data Center provides accelerometer data from a motor test rigwith single-point faults seeded by electro-discharge machining [1]. This work uses the 12 kHz drive-end data and the normal baseline files. Each .mat file holds a long drive-end record (key ending _DE_time), which is cut into overlapping windows; every window inherits the label of its file. The normal baseline files are recorded at 48 kHz and are decimated by 4 to 12 kHz so that all ten classes share one sampling rate.

Overlapping windows from the same recording are almost identical, so a randomtrain/test split puts near-duplicates on both sides and inflates accuracy [2]. The split is therefore made by motor load: train on 0, 1 and 2 hp, test on 3 hp, so that test recordings are never seen in training. Table 2 lists the dataset after windowing.

Item	Value
Classes	10
Window length / step	2048 / 1024 samples
Total windows	4643
Training windows (0, 1, 2 hp)	3465
Test windows (3 hp)	1178

Table 2: Dataset sizes after windowing

Optionaldenoisingappliessoftthresholdingwiththeuniversalthresholdtothedetail coefficients, which corresponds to the wavelet denoising experiment of the laboratory.

2.6 Model Implementation

Wavelet decomposition: a 5-level DWT at fs = 12 kHz divides 0–6 kHz intoone approximation band and five detail bands (Figure 2). Bearing impacts excite structural resonances at higher frequencies, so cD1–cD3 are expected to carry most fault information.

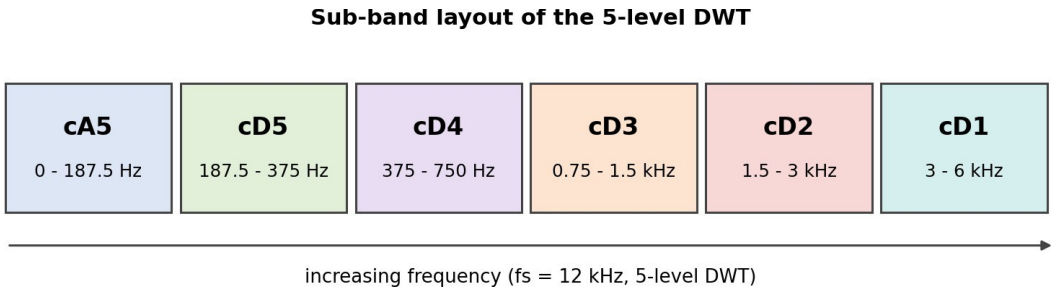


Figure 2: Sub-band layout of the 5-level DWT at 12 kHz sampling

Features (37): relative energy of the 6 sub-bands (6); mean absolute value,standard deviation, kurtosis, skewness and peak of each sub-band (30); and wavelet entropy (1).

Classifiers: Random Forest (300 trees), XGBoost (300 rounds, depth 6, learning rate0.1) and an RBF-SVM on standardised features. Hyper-parameters are tuned by load-grouped cross-validation on the training loads only. The values above are the starting configuration; the grouped cross-validation selected max_depth = None and min_samples_leaf = 3 for Random Forest, max_depth = 4 and learning rate 0.05 for XGBoost, and C = 1 and gamma = 0.01 for the SVM.

3 Results and Discussion

3.1 Classification Accuracy on the Test Set

Classifier	Features	Test accuracy	Macro F1
Random Forest	Wavelet (37)	99.1%	0.991
XGBoost	Wavelet (37)	97.9%	0.979
RBF-SVM	Wavelet (37)	99.1%	0.991
Random Forest	Time-domain only (8)	96.7%	0.967

Table 3: Test accuracy on the held-out 3 hp load

The leave-one-load-out mean and standard deviation over the four loads are also reported, because a single split can be lucky. Random Forest reaches 97.9 ± 2.5%, XGBoost 95.9 ± 4.8% and the RBF-SVM 94.0 ± 6.0% (mean ± standard deviation).

3.2 Per-Class Performance

Strongest classes (F1)	Weakest classes (F1)
OR_021 (1.000)	OR_014 (0.956)
OR_007 (1.000)	Ball_014 (0.975)
Normal (1.000)	Ball_021 (0.979)

Table 4: Strongest and weakest classes by F1-score

Seven of the ten classes (normal, all three inner-race sizes, and outer-race 0.007 and 0.021 inch) are classified perfectly (F1 = 1.000). All errors are confined to ball faults and the 0.014 inch outer-race fault: 10 of the 118 OR_014 test windows were predicted as ball faults (6 as Ball_014 and 4 as Ball_021) and one Ball_007 window was predicted as Ball_021 (Figure 3). This agrees with the benchmark study [2], which found ball faults among the hardest classes, although the smallest fault size was not the weakest here.

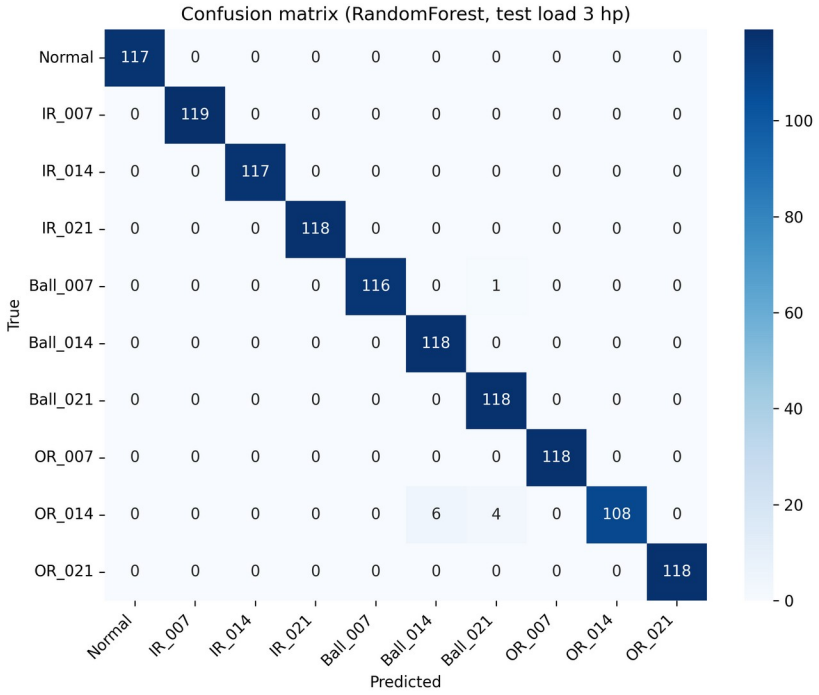


Figure 3: Confusion matrix of the best model on the held-out load

3.3 Feature-Set and Wavelet Ablation

Mother wavelet	Wavelet features only (accuracy)	Time + wavelet (accuracy)
Haar	96.9%	95.3%
db2	98.4%	98.3%
db4	99.1%	98.6%
db8	99.8%	99.2%

Table 5: Effect of the mother wavelet and the feature set (Random Forest)

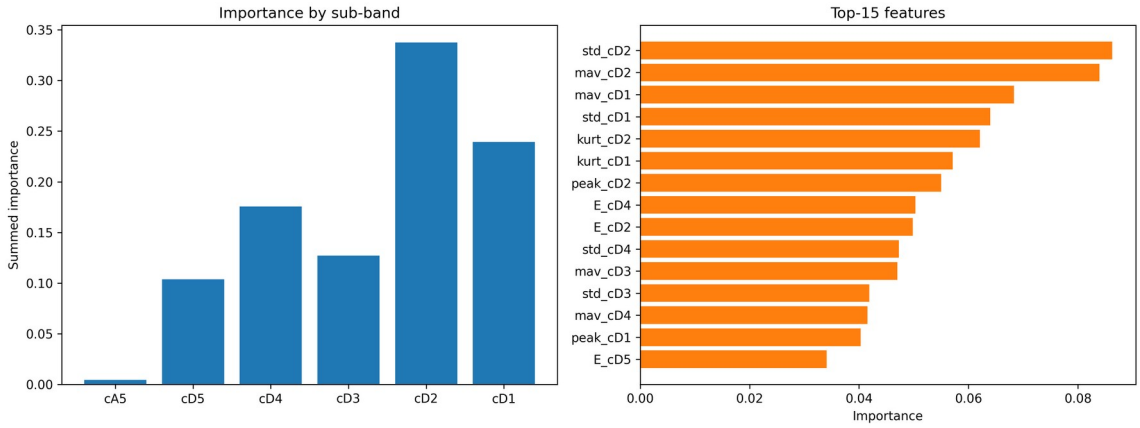


Figure 4: Random Forest feature importance by sub-band

3.4 Discussion

Wavelet versus time-domain features. On the unseen 3 hp load the Random Forest reaches 99.1% accuracy (macro F1 0.991) with the 37 wavelet features against 96.7% (0.967) with the 8 time-domain features, which cuts the error from 3.3% to 0.9%. Random Forest and the RBF-SVM tie at 99.1% and XGBoost reaches 97.9%. Appending the time-domain features to the wavelet set did not help (for db4, 98.6% against 99.1%), so the sub-band descriptors already carry the information. Accuracy rose with the length of the mother wavelet (Haar 96.9%, db2 98.4%, db4 99.1%, db8 99.8%); the db4 to db8 gap is about 9 of the 1178 test windows, so db4 is a reasonable default while db8 was the best choice in this test.

Important sub-bands. The summed Random Forest importance is highest for cD2 (1.5-3 kHz, about 0.34) and cD1 (3-6 kHz, about 0.24), followed by cD4 (0.18), cD3 (0.13) and cD5 (0.10), while the low-frequency approximation cA5 contributes almost nothing (0.004); cD2 and cD1 together account for roughly 0.58 of the total importance (Figure 4). The five most important individual features are std_cD2, mav_cD2, mav_cD1, std_cD1 and kurt_cD2. This supports the expectation that bearing impacts show up in the high-frequency detail bands, where the fault resonances lie.

Random split versus cross-load. A random 70/30 window split gives 99.4% for the Random Forest against 99.1% on the unseen 3 hp load, a drop of only 0.4 points, so on this dataset the leakage effect is small for tree models. The cross-load result is still the more honest figure, and a single split can flatter the model: in leave-one-load-out testing the Random Forest ranges from 93.6% (0 hp held out) to 99.9% (2 hp), and the SVM falls to 84.9% at 0 hp. The Random Forest is the most stable (97.9 ± 2.5%). Note that the best model was picked by test accuracy and that Random Forest and SVM tie on the 3 hp split.

Why not deep learning? A deep model is not required here. The feature pipeline trains in seconds on a CPU, needs little data and is explainable, because feature importance maps directly to frequency bands and physical fault signatures. Deep learning can stillbe benchmarked on the same split, for example a 1-D CNN on raw windows, to show the accuracy-versus-cost trade-off.

3.5 Conclusion

This project demonstrates a wavelet-based predictive-maintenance pipeline on theCWRUbearing benchmark. Multi-resolution DWT features with a tree-ensemble classifier reach 99.1%accuracyonanunseenmotorload,andsub-bandimportanceshowswhich frequencybandsrevealfaults.ThelimitsarethatCWRUisalaboratorydatasetwith seeded faults, that a fixed mother wavelet and level are assumed, and that results may not transfertoothermachineswithoutadaptation.Overall,waveletfeatureswithatree ensemble form a fast, explainable condition-monitoring baseline.

3.6 Future Scope

The current system provides a fast and explainable solution; however, it can be extended in several ways:

- Use **wavelet packet decomposition** and automatic selection of the most informative sub-bands.
- Add **envelope analysis** and compare with a **1-D CNN or scalogram CNN** onthe same split.
- Test on other bearing datasets (for example Paderborn) and under added noise.
- Deploy on an **embedded board** for real-time monitoring and add remaining-useful-life estimation.

References

[1] Case Western Reserve University Bearing Data Center, "Bearing Data Center," Case SchoolofEngineering.
Reference link: https://engineering.case.edu/bearingdatacenter

[2] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," Mechanical Systems and Signal Processing, vol. 64–65, pp. 100–131, 2015, doi: 10.1016/j.ymssp.2015.04.021.
Reference link: https://doi.org/10.1016/j.ymssp.2015.04.021

[3] S. G. Mallat, "A theory for multiresolution signal decomposition: the wavelet representation,"IEEETrans. Pattern Analysis and Machine Intelligence, vol. 11, no. 7, pp. 674–693, 1989, doi: 10.1109/34.192463.
Reference link: https://doi.org/10.1109/34.192463

[4] I. Daubechies, Ten Lectures on Wavelets, SIAM, 1992, doi: 10.1137/1.9781611970104.
Reference link: https://doi.org/10.1137/1.9781611970104

[5] Y. Lei et al., "Applications of machine learning to machine fault diagnosis: A review androadmap,"Mechanical Systems and Signal Processing, vol. 138, art. 106587, 2020, doi: 10.1016/j.ymssp.2019.106587.
Reference link: https://doi.org/10.1016/j.ymssp.2019.106587

[6] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5–32, 2001,doi: 10.1023/A:1010933404324; T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," Proc. ACM KDD, pp. 785–794, 2016, doi: 10.1145/2939672.2939785.

Reference link: https://doi.org/10.1145/2939672.2939785

[7] R. M. Rao and A. S. Bopardikar, Wavelet Transforms: Introduction to Theory and Applications, Pearson Education, Asia, 2000.

[8] J. G. Proakis and D. G. Manolakis, Digital Signal Processing, PHI, 2014.

Appendix A: Python Code

The complete implementation was developed in Google Colab using Python,PyWavelets, scikit-learn and XGBoost. Upload the CWRU .mat files, named by file number (for example 105.mat), to a folder called cwru, and run `pip install PyWavelets xgboost` first. The code is listed cell by cell in the order in which it is run.

Cell 1: Libraries and configuration

```
import os, time, json, warnings, joblib
from functools import partial
import numpy as np, scipy.io as sio, pywt, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy.stats import kurtosis, skew
from scipy.signal import decimate, stft
from joblib import Parallel, delayed
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GroupKFold, GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
from xgboost import XGBClassifier
warnings.filterwarnings("ignore")

DATA_DIR, OUT_DIR = "cwru", "results"
FS, WIN, STEP = 12000, 2048, 1024
WAVELET, LEVEL, DENOISE, SEED = "db4", 5, False, 42
NORMAL_FS = 48000   # files 97-100 are 48 kHz on the CWRU site; decimated by 4 to 12 kHz
TUNE, TEST_LOAD = True, 3
os.makedirs(OUT_DIR, exist_ok=True)
```

Cell 2: File map (12 kHz drive end), labels and data loading

```
# file numbers for loads 0, 1, 2, 3 hp
FILES = {
 "Normal":   [97, 98, 99, 100],
 "IR_007":   [105, 106, 107, 108], "IR_014":   [169, 170, 171, 172], "IR_021":   [209, 210, 211, 212],
 "Ball_007": [118, 119, 120, 121], "Ball_014": [185, 186, 187, 188], "Ball_021": [222, 223, 224, 225],
 "OR_007":   [130, 131, 132, 133], "OR_014":   [197, 198, 199, 200], "OR_021":   [234, 235, 236, 237]}
CLASSES = list(FILES.keys())

# optional: download the 40 .mat files into cwru/ (or upload them manually)
import urllib.request
os.makedirs(DATA_DIR, exist_ok=True)
URL = "https://engineering.case.edu/sites/default/files/{n}.mat"
for n in [n for v in FILES.values() for n in v]:
    dst = "%s/%d.mat" % (DATA_DIR, n)
    if not (os.path.exists(dst) and os.path.getsize(dst) > 10000):
        try: urllib.request.urlretrieve(URL.format(n=n), dst)
        except Exception as e: print("missing", n)

def load_de(num):
    m = sio.loadmat("%s/%d.mat" % (DATA_DIR, num))
    key = [k for k in m if k.endswith("_DE_time")][0]
    return m[key].ravel().astype(np.float64)

def windows(sig):
    n = (len(sig) - WIN) // STEP + 1
    return np.stack([sig[i*STEP:i*STEP + WIN] for i in range(n)])

def load_all_windows():
    W, y, load = [], [], []
    for ci, name in enumerate(CLASSES):
        for ld, num in enumerate(FILES[name]):
            sig = load_de(num)
            if name == "Normal" and NORMAL_FS == 48000:
                sig = decimate(sig, 4)
            w = windows(sig)
            W.append(w); y += [ci]*len(w); load += [ld]*len(w)
    return np.vstack(W), np.array(y), np.array(load)
```

Cell 3: Denoising and feature extraction

```
def denoise(x, wavelet=WAVELET, level=LEVEL):
    c = pywt.wavedec(x, wavelet, level=level)
    sigma = np.median(np.abs(c[-1])) / 0.6745
    thr = sigma * np.sqrt(2 * np.log(len(x)))
    c[1:] = [pywt.threshold(d, thr, mode="soft") for d in c[1:]]
    return pywt.waverec(c, wavelet)[:len(x)]

def wavelet_features(x, wavelet=WAVELET):
    if DENOISE: x = denoise(x, wavelet)
```

```
    c = pywt.wavedec(x, wavelet, level=LEVEL)              # [cA5, cD5, ..., cD1]
    e = np.array([np.sum(ci**2) for ci in c]); rel = e / e.sum()
    f = list(rel)
    for ci in c:
        f += [np.mean(np.abs(ci)), np.std(ci), kurtosis(ci), skew(ci), np.max(np.abs(ci))]
    p = rel + 1e-12
    f.append(-np.sum(p * np.log2(p)))                      # wavelet entropy
    return np.array(f)                                     # 6 + 30 + 1 = 37

def time_features(x):
    rms = np.sqrt(np.mean(x**2)); pk = np.max(np.abs(x))
    return np.array([x.mean(), x.std(), rms, pk, kurtosis(x), skew(x),
                     pk / rms, pk / np.mean(np.abs(x))])    # 8

def _chunk(fn, chunk):
    return np.array([fn(w) for w in chunk])

def featurize(fn, W, n_chunks=64):
    chunks = np.array_split(W, n_chunks)
    return np.vstack(Parallel(n_jobs=-1)(delayed(_chunk)(fn, c) for c in chunks if len(c)))
```

Cell 4: Models and load-grouped hyper-parameter tuning

```
def make_models(params=None):
    p = params or {}
    rf = {"n_estimators": 300, "random_state": SEED, "n_jobs": -1}; rf.update(p.get("RandomForest", {}))
    xg = {"n_estimators": 300, "max_depth": 6, "learning_rate": 0.1, "subsample": 0.8,
          "colsample_bytree": 0.8, "random_state": SEED, "n_jobs": -1}; xg.update(p.get("XGBoost", {}))
    sv = {"kernel": "rbf", "C": 10, "gamma": "scale"}; sv.update(p.get("SVM-RBF", {}))
    return {"RandomForest": RandomForestClassifier(**rf), "XGBoost": XGBClassifier(**xg),
            "SVM-RBF": make_pipeline(StandardScaler(), SVC(**sv))}

def tune_models(X, y, load, test_load):
    """Load-grouped CV on the TRAINING loads only (the test load is never used)."""
    tr = load != test_load
    gkf = GroupKFold(n_splits=len(np.unique(load[tr])))
    grids = {"RandomForest": {"max_depth": [None, 20], "min_samples_leaf": [1, 3]},
             "XGBoost": {"max_depth": [4, 6], "learning_rate": [0.05, 0.1]},
             "SVM-RBF": {"svc__C": [1, 10, 100], "svc__gamma": ["scale", 0.01]}}
    best = {}
    for name, m in make_models().items():
        gs = GridSearchCV(m, grids[name], cv=gkf, scoring="f1_macro", n_jobs=1)
        gs.fit(X[tr], y[tr], groups=load[tr])
        bp = {k.replace("svc__", ""): v for k, v in gs.best_params_.items()}
        best[name] = bp
        print("tuned %-12s %s (grouped-CV macroF1 %.4f)" % (name, bp, gs.best_score_))
    return best
```

Cell 5: Build the dataset (Table 2)

```
W, y, load = load_all_windows()
Xw = featurize(wavelet_features, W)          # 37 wavelet features
Xt = featurize(time_features, W)             # 8 time-domain features
tr, te = load != TEST_LOAD, load == TEST_LOAD
print(W.shape, Xw.shape, Xt.shape)
print("total", len(y), "train", tr.sum(), "test", te.sum())
print(dict(zip(CLASSES, np.bincount(y))))
```

Cell 6: Tuning and cross-load evaluation (Table 3, train 0, 1, 2 hp / test 3 hp)

```
params = tune_models(Xw, y, load, TEST_LOAD) if TUNE else None
res = {}
for name, m in make_models(params).items():
    t0 = time.time(); m.fit(Xw[tr], y[tr]); p = m.predict(Xw[te])
    res[name] = (accuracy_score(y[te], p), f1_score(y[te], p, average="macro"), p)
    print("%-13s acc %.4f macroF1 %.4f (%.1fs)" % (name, res[name][0], res[name][1], time.time() - t0))
rf_t = make_models(params)["RandomForest"].fit(Xt[tr], y[tr]); pt = rf_t.predict(Xt[te])
print("RF time-domain acc %.4f macroF1 %.4f" % (accuracy_score(y[te], pt),
      f1_score(y[te], pt, average="macro")))
```

Cell 7: Leave-one-load-out and random-split comparison

```
for name in make_models():
    accs = []
    for L in range(4):
        a, b = load != L, load == L
        m = make_models(params)[name].fit(Xw[a], y[a])
        accs.append(accuracy_score(y[b], m.predict(Xw[b])))
    print(name, np.round(accs, 4), "mean %.4f std %.4f" % (np.mean(accs), np.std(accs)))

Xa, Xb, ya, yb = train_test_split(Xw, y, test_size=0.3, stratify=y, random_state=SEED)
rand_acc = accuracy_score(yb, make_models(params)["RandomForest"].fit(Xa, ya).predict(Xb))
print("random 70/30 split %.4f vs cross-load %.4f" % (rand_acc, res["RandomForest"][0]))
```

Cell 8: Per-class report (Table 4) and confusion matrix (Figure 3)

```
best = max(res, key=lambda k: res[k][0]); p_best = res[best][2]
print(best); print(classification_report(y[te], p_best, target_names=CLASSES, digits=3))
plt.figure(figsize=(8, 6.5))
sns.heatmap(confusion_matrix(y[te], p_best), annot=True, fmt="d", cmap="Blues",
            xticklabels=CLASSES, yticklabels=CLASSES)
plt.xlabel("Predicted"); plt.ylabel("True")
plt.title("Confusion matrix (%s, test load %d hp)" % (best, TEST_LOAD))
plt.xticks(rotation=45, ha="right"); plt.tight_layout()
```

```
plt.savefig(OUT_DIR + "/fig3_confusion_matrix.png", dpi=300); plt.show()
```

Cell 9: Feature-set and mother-wavelet ablation (Table 5)

```
rf = lambda: make_models(params)["RandomForest"]
for wv in ["haar", "db2", "db4", "db8"]:
    Xv = featurize(partial(wavelet_features, wavelet=wv), W)
    a1 = accuracy_score(y[te], rf().fit(Xv[tr], y[tr]).predict(Xv[te]))
    Xc = np.hstack([Xv, Xt])
    a2 = accuracy_score(y[te], rf().fit(Xc[tr], y[tr]).predict(Xc[te]))
    print("%-5s wavelet %.4f | time+wavelet %.4f" % (wv, a1, a2))
```

Cell 10: Feature importance by sub-band (Figure 4) and model saving

```
bands = ["cA5", "cD5", "cD4", "cD3", "cD2", "cD1"]
# order matches wavelet_features: 6 energies, then 5 statistics per band, then entropy
names = (["E_" + b for b in bands]
         + [s + "_" + b for b in bands for s in ["mav", "std", "kurt", "skew", "peak"]] + ["entropy"])
rf_m = make_models(params)["RandomForest"].fit(Xw[tr], y[tr])
imp = rf_m.feature_importances_
band_imp = {b: sum(imp[i] for i, n in enumerate(names) if n.endswith("_" + b)) for b in bands}
top = np.argsort(imp)[::-1][:15]
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].bar(band_imp.keys(), band_imp.values(), color="tab:blue")
ax[0].set_ylabel("Summed importance"); ax[0].set_title("Importance by sub-band")
ax[1].barh([names[i] for i in top][::-1], imp[top][::-1], color="tab:orange")
ax[1].set_xlabel("Importance"); ax[1].set_title("Top-15 features")
plt.tight_layout(); plt.savefig(OUT_DIR + "/fig4_feature_importance.png", dpi=300); plt.show()
print({k: round(v, 3) for k, v in band_imp.items()})
joblib.dump(rf_m, OUT_DIR + "/bearing_rf.pkl")
```

Cell 11: STFT versus FT versus DWT demonstration (laboratory link)

```
sig = load_de(FILES["IR_014"][0])[:WIN * 4]
f_, t_, Z = stft(sig, fs=FS, nperseg=256)
c = pywt.wavedec(sig, WAVELET, level=LEVEL)
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ax[0].plot(np.fft.rfftfreq(len(sig), 1 / FS), np.abs(np.fft.rfft(sig))); ax[0].set_title("FT magnitude")
ax[1].pcolormesh(t_, f_, np.abs(Z), shading="auto"); ax[1].set_title("STFT")
ax[2].bar(bands, [np.sum(ci**2) for ci in c]); ax[2].set_title("DWT sub-band energy")
plt.tight_layout(); plt.show()
```